In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import cophenet, dendrogram, fcluster, linkage
from scipy.spatial.distance import pdist
from sklearn.cluster import AgglomerativeClustering
from sklearn.decomposition import PCA
import os
os.environ["OMP_NUM_THREADS"] = "3"
```

In [2]:
```
df = pd.read_excel('C:/Users/ADMIN/OneDrive/Documents/Credit Card Customer Dat
```

In [3]:
```
df.head()
```

Out[3]:

	Sl_No	Customer Key	Avg_Credit_Limit	Total_Credit_Cards	Total_visits_bank	Tota
0	1	87073	100000	2	1	
1	2	38414	50000	3	0	
2	3	17341	50000	7	1	
3	4	40496	30000	5	1	
4	5	47437	100000	6	0	

In [4]:
```
df.describe()
```

Out[4]:

	Sl_No	Customer Key	Avg_Credit_Limit	Total_Credit_Cards	Total_visit
count	660.000000	660.000000	660.000000	660.000000	660
mean	330.500000	55141.443939	34574.242424	4.706061	2
std	190.669872	25627.772200	37625.487804	2.167835	1
min	1.000000	11265.000000	3000.000000	1.000000	0
25%	165.750000	33825.250000	10000.000000	3.000000	1
50%	330.500000	53874.500000	18000.000000	5.000000	2
75%	495.250000	77202.500000	48000.000000	6.000000	4
max	660.000000	99843.000000	200000.000000	10.000000	5

In [5]:
```
print("Missing values in columns:")
df.isnull().sum()
```

```
Missing values in columns:
```

Out[5]:
```
Sl_No                  0
Customer Key           0
Avg_Credit_Limit       0
Total_Credit_Cards     0
Total_visits_bank      0
Total_visits_online    0
Total_calls_made       0
dtype: int64
```

In [6]:
```
df.duplicated().sum()
```

Out[6]:
```
0
```

In [7]:
```
gb_df = df.groupby('Customer Key').count
```

In [8]:
```
df_duplicates = df[df.duplicated(subset=['Customer Key'], keep=False)]
df_duplicates.sort_values(by='Customer Key')
```

Out[8]:

	Sl_No	Customer Key	Avg_Credit_Limit	Total_Credit_Cards	Total_visits_bank	To
48	49	37252	6000	4	0	
432	433	37252	59000	6	2	
4	5	47437	100000	6	0	
332	333	47437	17000	7	3	
411	412	50706	44000	4	5	
541	542	50706	60000	7	5	
391	392	96929	13000	4	5	
398	399	96929	67000	6	2	
104	105	97935	17000	2	1	
632	633	97935	187000	7	1	

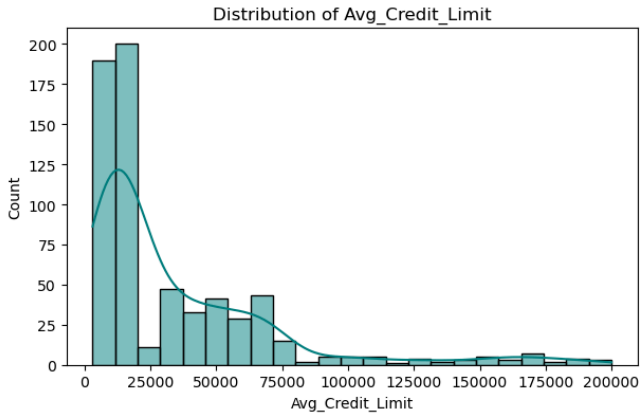
In [9]:
```
unique_sl = df['Sl_No'].nunique()
unique_key = df['Customer Key'].nunique()
print(f"Unique Sl_No: {unique_sl}")
print(f"Unique Customer Key: {unique_key}")
print(f"Total rows: {len(df)}")
```

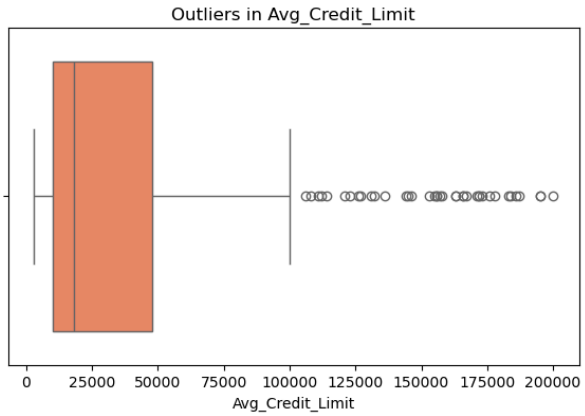
```
Unique Sl_No: 660
Unique Customer Key: 655
Total rows: 660
```

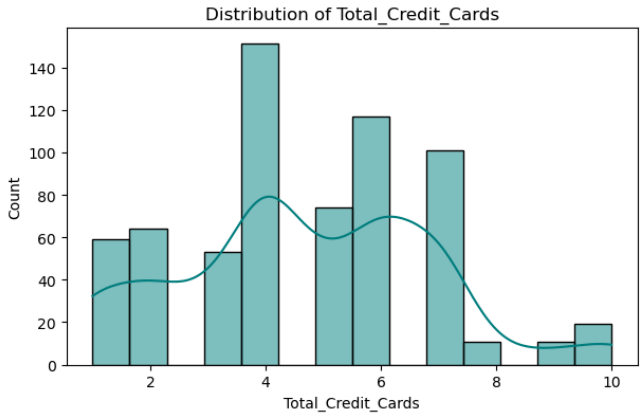
In [10]:
```
df_clean = df.drop_duplicates(subset='Customer Key').copy()
data = df.drop(['Sl_No', 'Customer Key'], axis=1)
```

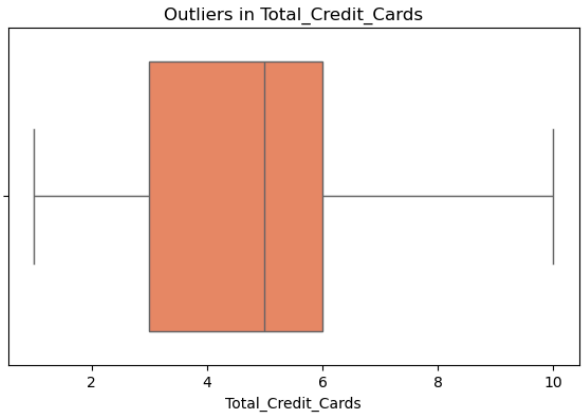
In [11]:
```
features = data.columns
```

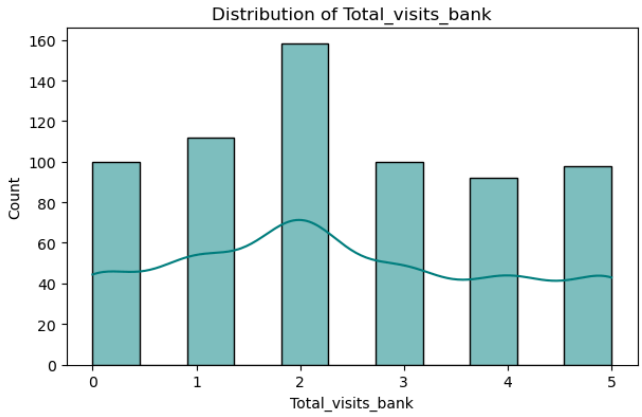
```
fig, axes = plt.subplots(len(features), 2, figsize=(12, 20))

for i, col in enumerate(features):
    sns.histplot(data[col], kde=True, ax=axes[i, 0], color='teal')
    axes[i, 0].set_title(f'Distribution of {col}', fontsize=12)
    sns.boxplot(x=data[col], ax=axes[i, 1], color='coral')
    axes[i, 1].set_title(f'Outliers in {col}', fontsize=12)

plt.tight_layout()
plt.savefig('preprocessing_boxplots.png')
```

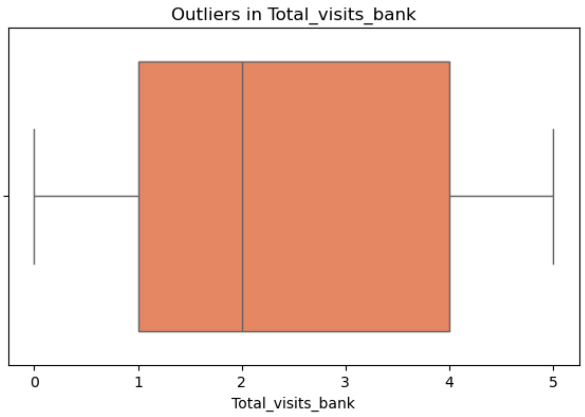

Distribution of Avg_Credit_Limit
Count
Avg_Credit_Limit

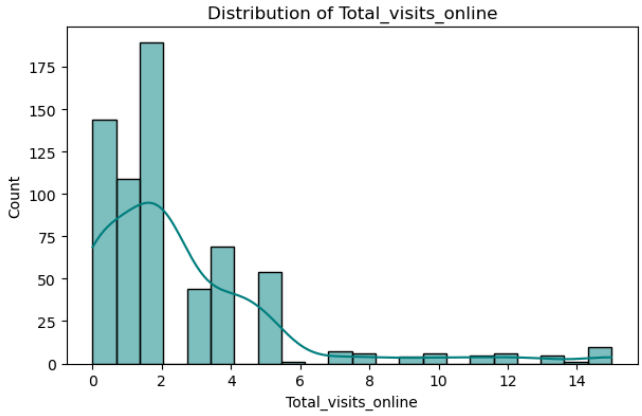

Outliers in Avg_Credit_Limit
Avg_Credit_Limit

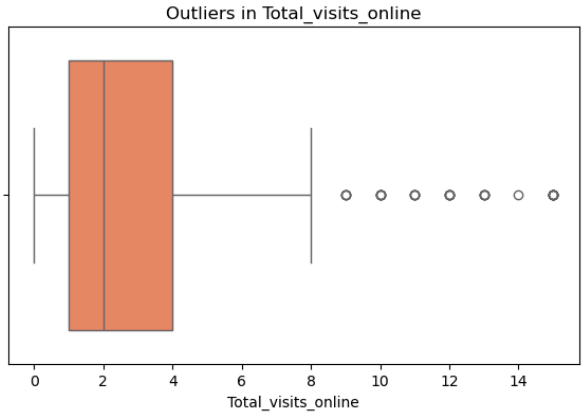

Distribution of Total_Credit_Cards
Count
Total_Credit_Cards

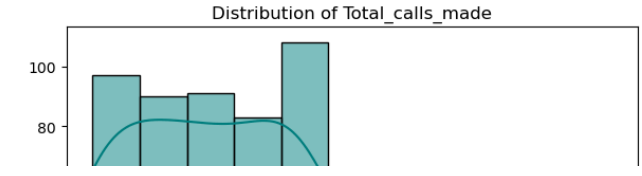

Outliers in Total_Credit_Cards
Total_Credit_Cards

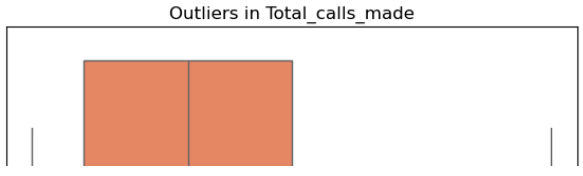

Distribution of Total_visits_bank
Count
Total_visits_bank


Outliers in Total_visits_bank
Total_visits_bank

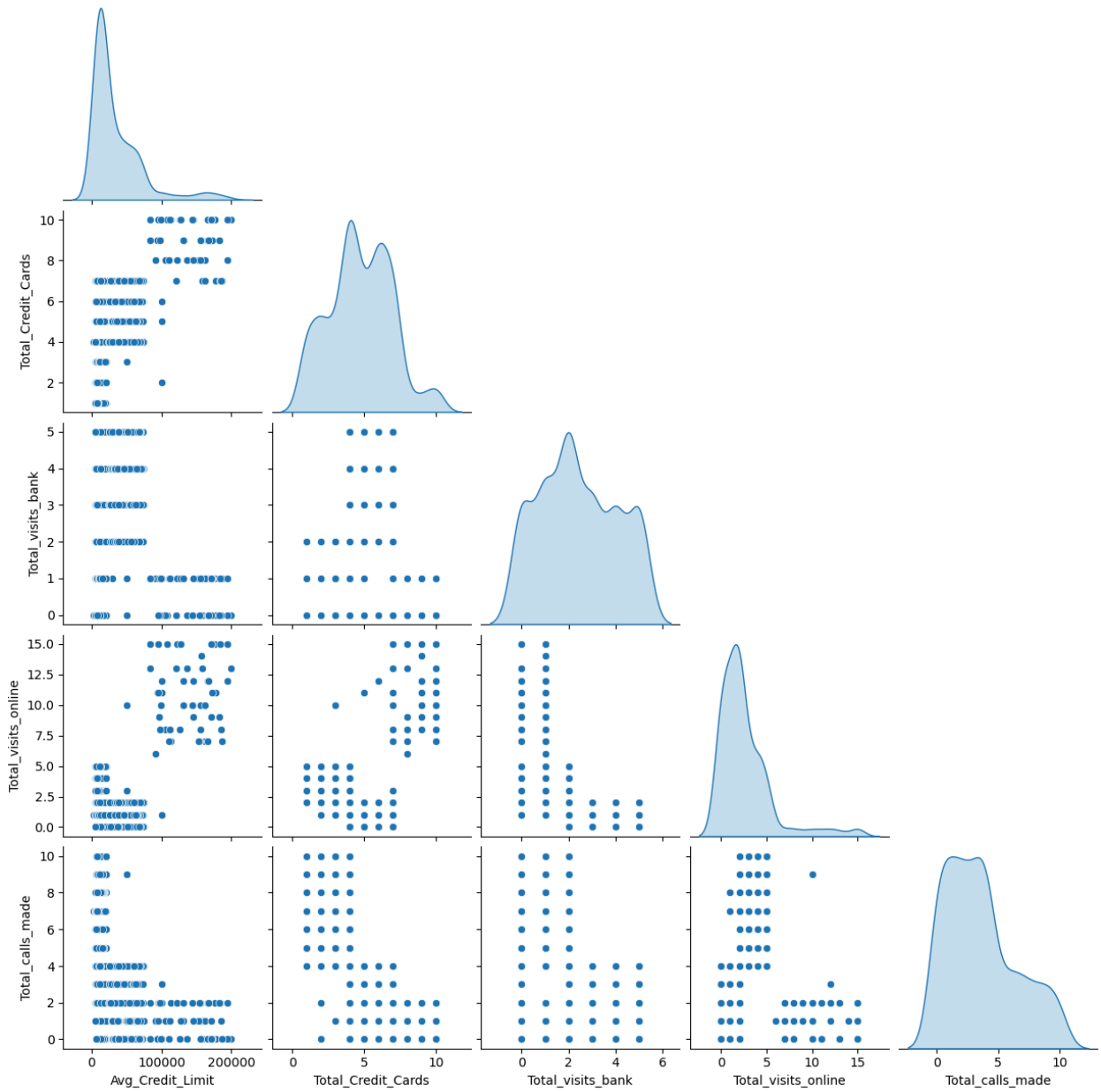

Distribution of Total_visits_online
Count
Total_visits_online


Outliers in Total_visits_online
Total_visits_online

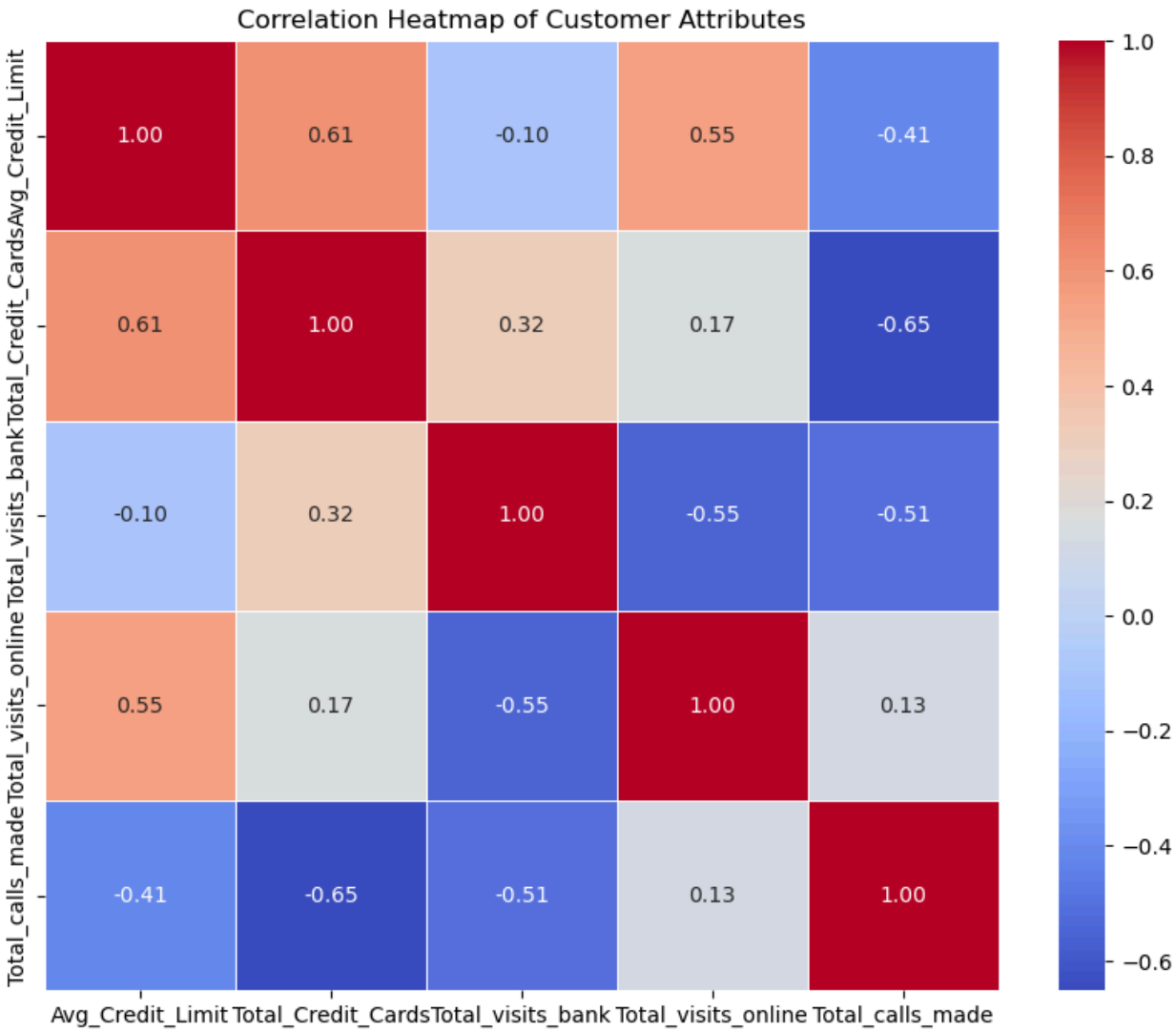

Distribution of Total_calls_made


Outliers in Total_calls_made

In [12]:
```
sns.pairplot(data, diag_kind='kde', corner=True)
plt.savefig('pairplot_analysis.png')
```



In [13]:
```
plt.figure(figsize=(10, 8))
corr = data.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Correlation Heatmap of Customer Attributes')
plt.savefig('correlation_heatmap.png')
```



In [14]:
```
df_proc = df.drop(['Sl_No', 'Customer Key'], axis=1)
df_proc['Total_Interactions'] = df_proc['Total_visits_bank'] + df_proc['Total_
```

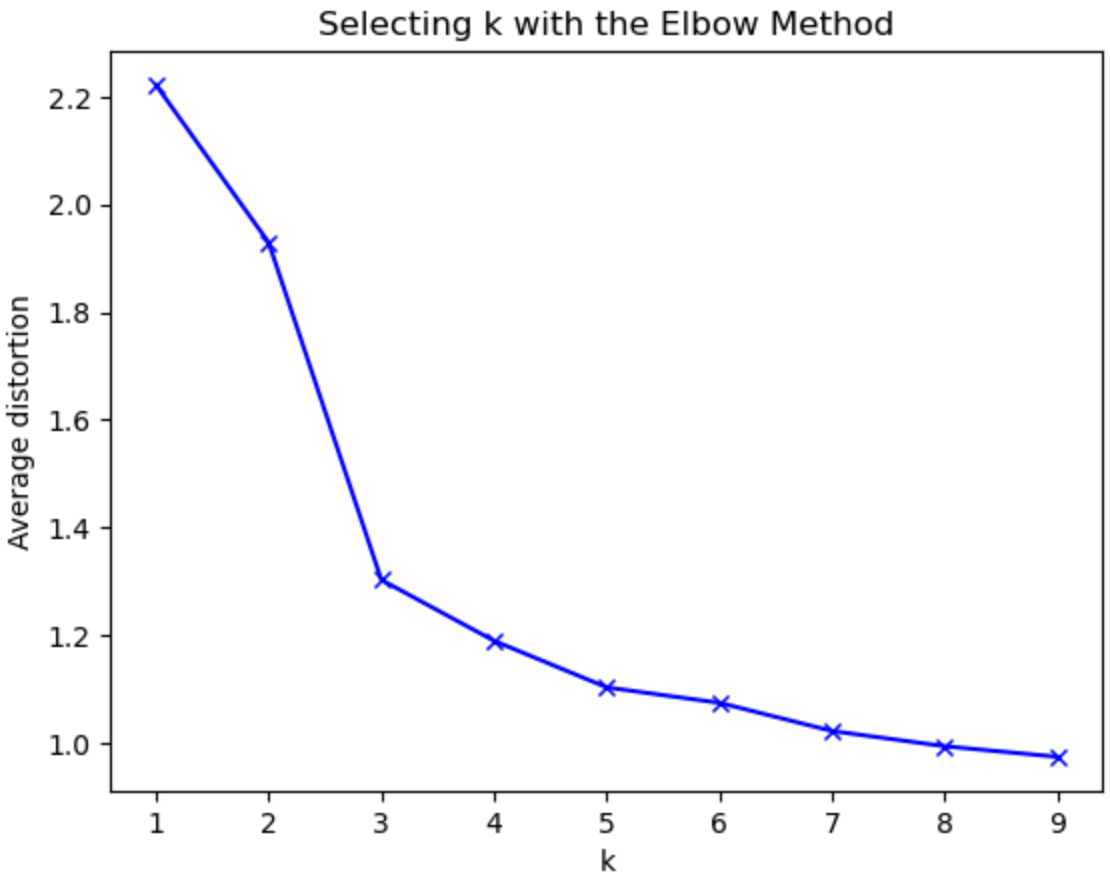
In [15]:
```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_proc)
df_scaled = pd.DataFrame(scaled_data, columns=df_proc.columns)

print("\nFirst 5 rows of Scaled Data:")
print(df_scaled.head())

df_proc.to_csv('preprocessed_data.csv', index=False)
df_scaled.to_csv('scaled_data.csv', index=False)
```

```
First 5 rows of Scaled Data:
   Avg_Credit_Limit  Total_Credit_Cards  Total_visits_bank  \
0          1.740187           -1.249225          -0.860451
1          0.410293           -0.787585          -1.473731
2          0.410293            1.058973          -0.860451
3         -0.121665            0.135694          -0.860451
4          1.740187            0.597334          -1.473731

   Total_visits_online  Total_calls_made  Total_Interactions
0            -0.547490         -1.251537           -1.935936
1             2.520519          1.891859            3.056296
2             0.134290          0.145528           -0.173972
3            -0.547490          0.145528           -0.761293
4             3.202298         -0.203739            1.881653
```

In [16]:

```
from sklearn.cluster import KMeans
from scipy.spatial.distance import cdist
meanDistortions=[]
k_range = range(1, 10)
sse = []
silhouette_avg = []
for k in k_range:

    model=KMeans(n_clusters=k)
    model.fit(df_scaled)
    prediction=model.predict(df_scaled)
    meanDistortions.append(sum(np.min(cdist(df_scaled, model.cluster_centers_,


plt.plot(range(1, 10), meanDistortions, 'bx-')
plt.xlabel('k')
plt.ylabel('Average distortion')
plt.title('Selecting k with the Elbow Method');
```

```
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
```



In [17]:
```
model = KMeans(n_clusters=3)
model.fit(df_scaled)
preds = model.predict(df_scaled)
from sklearn.metrics import silhouette_score
labels = model.labels_
silhouette_score(df_scaled, labels, metric='euclidean')
```

```
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
```

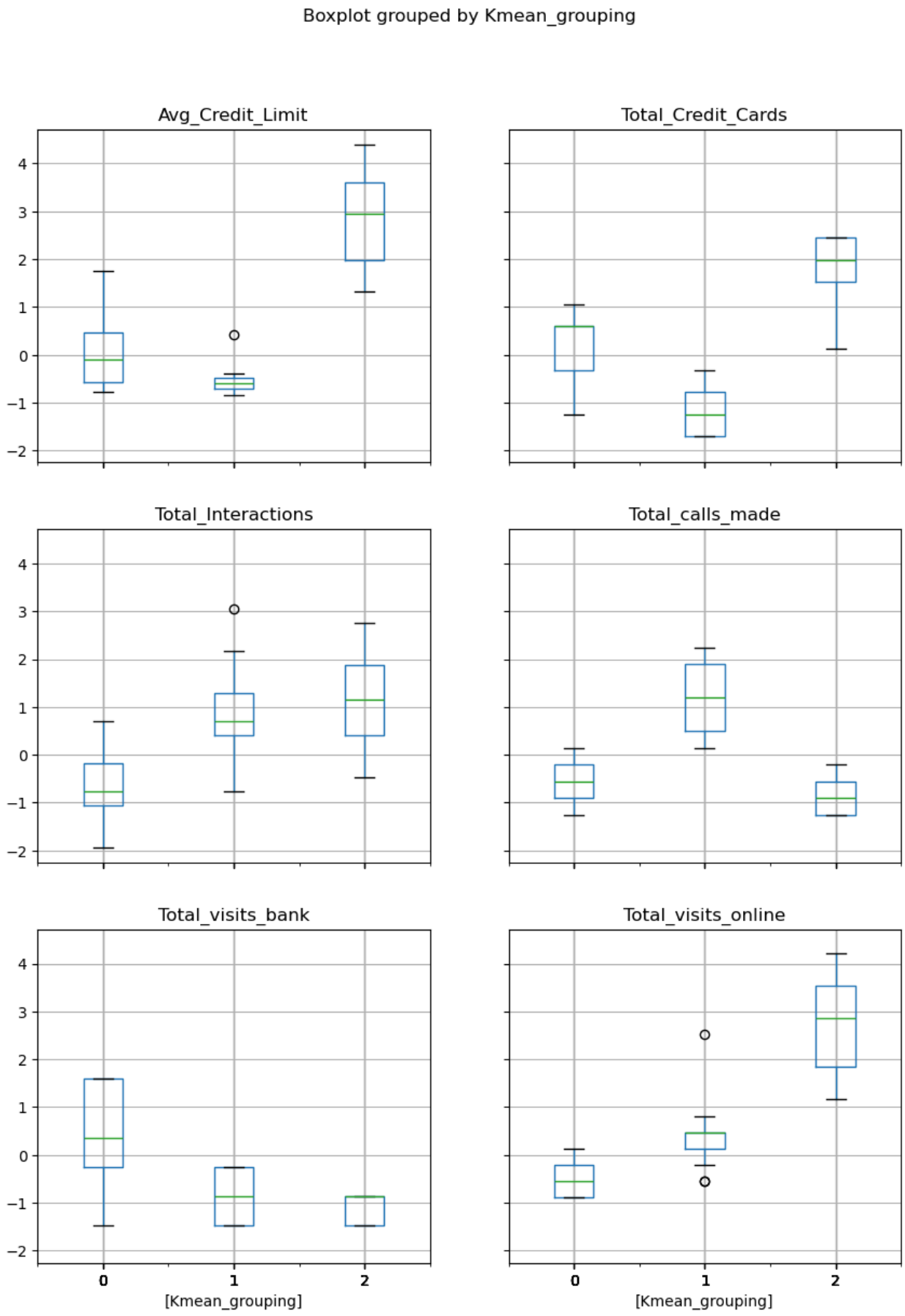
Out[17]: 0.5084388494134671

In [18]:
```
df['Kmean_grouping'] = preds
df_scaled['Kmean_grouping'] = preds

df.groupby('Kmean_grouping').count()
```

Out[18]:

Kmean_grouping	Sl_No	Customer Key	Avg_Credit_Limit	Total_Credit_Cards	Total_v
0	389	389	389	389	
1	221	221	221	221	
2	50	50	50	50	

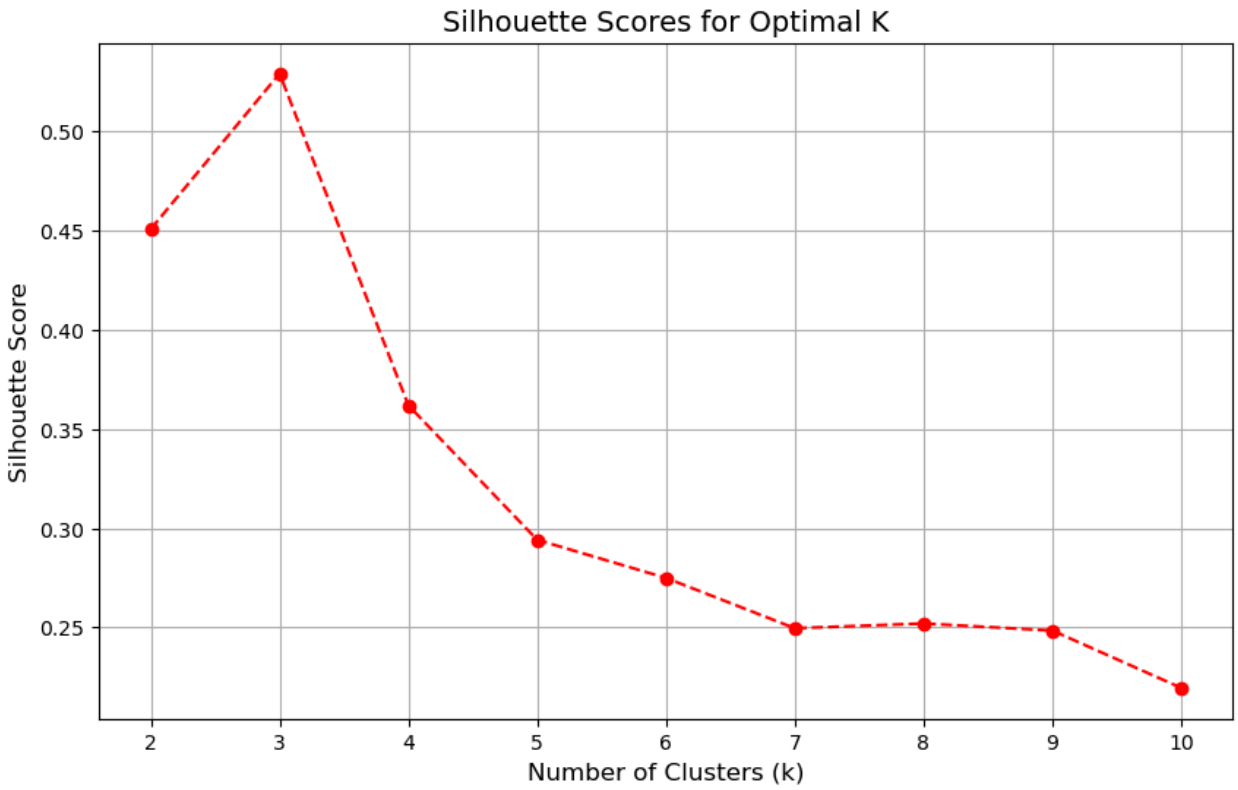
In [19]:
```
df_scaled.boxplot(by='Kmean_grouping', layout=(3,2), figsize=(10,14));
```


Boxplot grouped by Kmean_grouping
Avg_Credit_Limit
Total_Credit_Cards
Total_Interactions
Total_calls_made
Total_visits_bank
Total_visits_online
4
3
2
1
0
−1
−2
0
1
2
[Kmean_grouping]

In [20]:

```
sse = []
silhouette_avg = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(df_scaled)
    sse.append(kmeans.inertia_)
    silhouette_avg.append(silhouette_score(df_scaled, kmeans.labels_))
plt.figure(figsize=(10, 6))
plt.plot(k_range, silhouette_avg, marker='o', linestyle='--', color='r')
plt.title('Silhouette Scores for Optimal K', fontsize=14)
plt.xlabel('Number of Clusters (k)', fontsize=12)
plt.ylabel('Silhouette Score', fontsize=12)
plt.xticks(k_range)
plt.grid(True)
plt.savefig('silhouette_score_only.png')
```

```
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
```



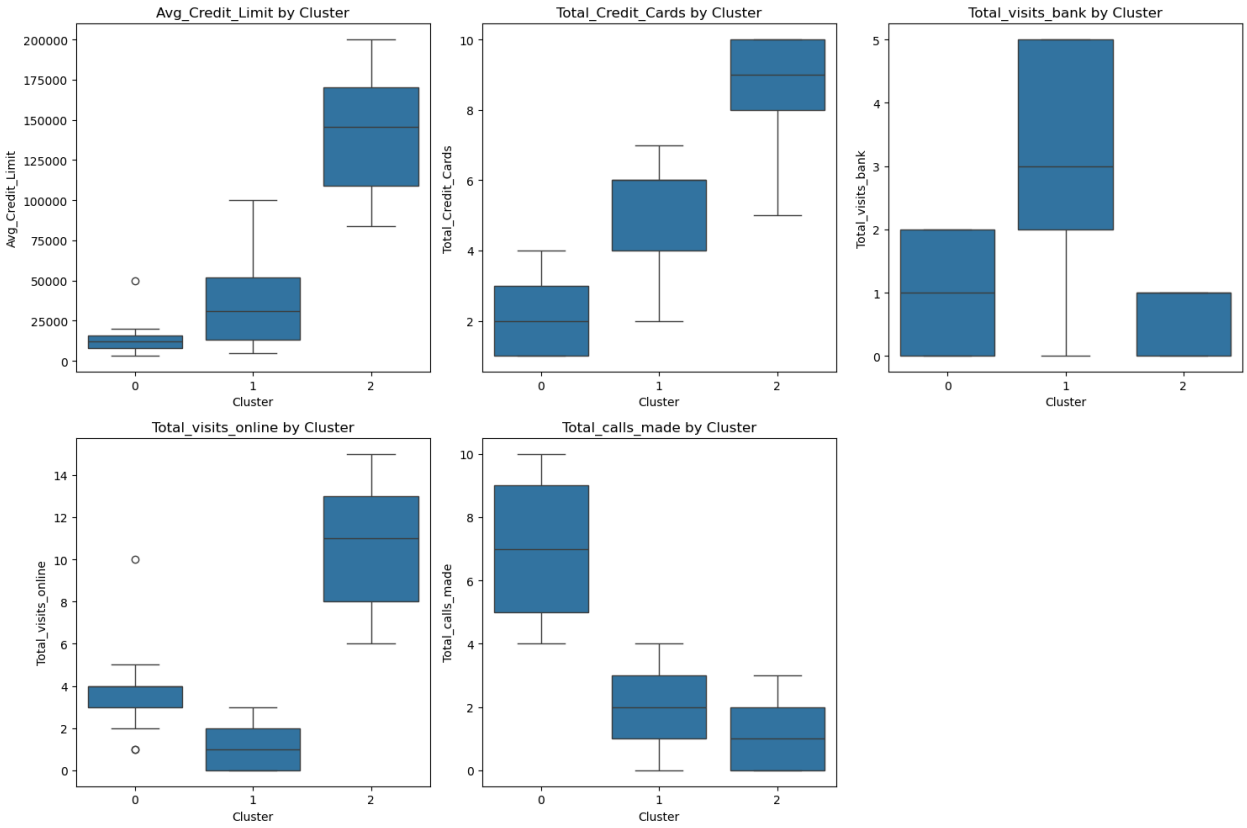
In [21]:
```
df_scaled = pd.read_csv('scaled_data.csv')
df_orig = pd.read_csv('preprocessed_data.csv')
optimal_k = 3
kmeans_final = KMeans(n_clusters=optimal_k)
df_orig ['Cluster'] = kmeans_final.fit_predict(df_scaled)
cluster_profile = df_orig.groupby('Cluster').mean()
print("\nCluster Profiles (Mean values):")
print(cluster_profile)
```

```
Cluster Profiles (Mean values):
         Avg_Credit_Limit  Total_Credit_Cards  Total_visits_bank  \
Cluster
0             12217.19457            2.402715           0.936652
1             33591.25964            5.496144           3.467866
2            141040.00000            8.740000           0.600000

         Total_visits_online  Total_calls_made  Total_Interactions
Cluster
0                   3.579186          6.932127           11.447964
1                   0.987147          2.002571            6.457584
2                  10.900000          1.080000           12.580000
```

```
C:\Users\ADMIN\.anaconda\ANN\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when th
ere are less chunks than available threads. You can avoid it by setting the env
ironment variable OMP_NUM_THREADS=3.
  warnings.warn(
```

In [22]:
```
plt.figure(figsize=(15, 10))
features = ['Avg_Credit_Limit', 'Total_Credit_Cards', 'Total_visits_bank', 'To
```

```
for i, feature in enumerate(features):
    plt.subplot(2, 3, i+1)
    sns.boxplot(x='Cluster', y=feature, data=df_orig)
    plt.title(f'{feature} by Cluster')

plt.tight_layout()
plt.savefig('cluster_boxplots.png')
```

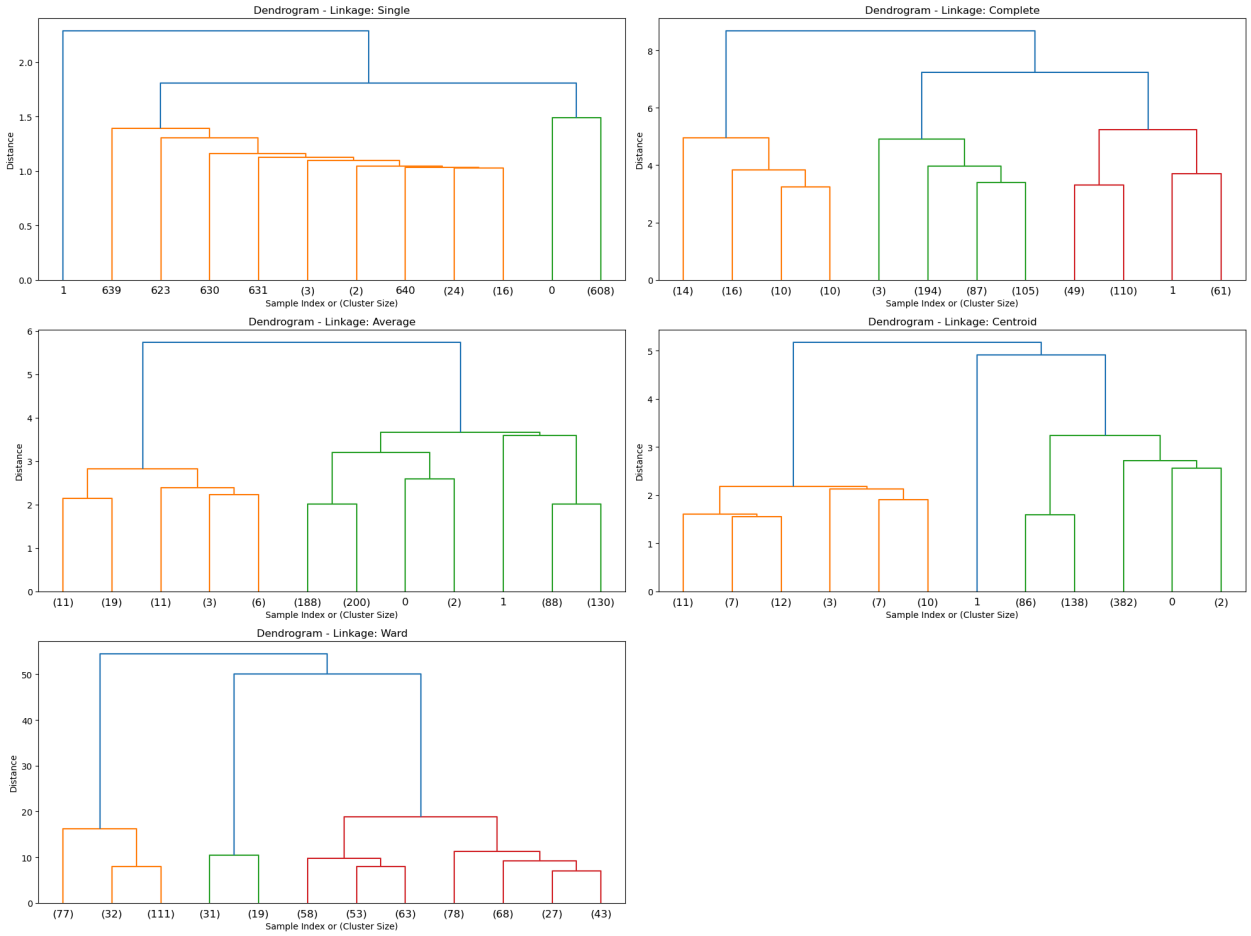


In [23]:
```
methods = ['single', 'complete', 'average', 'centroid', 'ward']
cophenetic_results = {}

dist_matrix = pdist(df_scaled)

for method in methods:
    Z = linkage(df_scaled, method=method)
    c, coph_dists = cophenet(Z, dist_matrix)
    cophenetic_results[method] = c
for method, coeff in cophenetic_results.items():
    print(f"{method.capitalize()} Linkage: {coeff:.4f}")
```

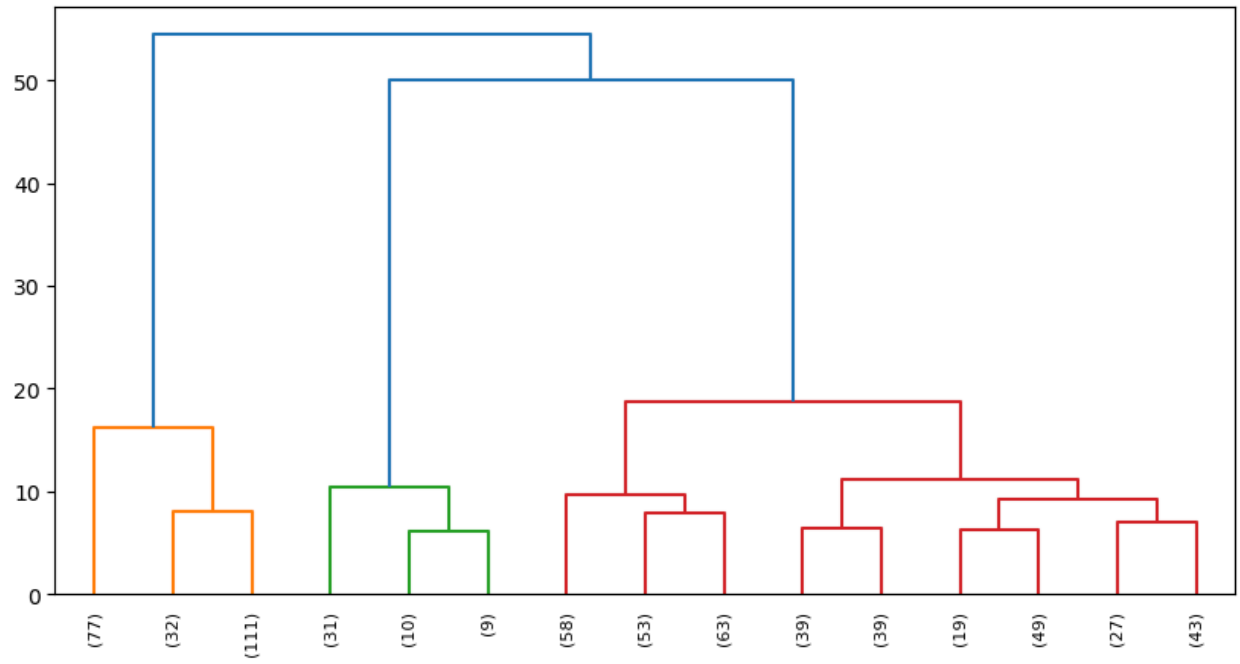
```
Single Linkage: 0.6993
Complete Linkage: 0.8306
Average Linkage: 0.8789
Centroid Linkage: 0.8742
Ward Linkage: 0.7466
```

In [24]:
```
plt.figure(figsize=(20, 15))

for i, method in enumerate(methods):
```

```
    Z = linkage(df_scaled, method=method)
    plt.subplot(3, 2, i+1)
    plt.title(f'Dendrogram - Linkage: {method.capitalize()}')
    dendrogram(Z, truncate_mode='lastp', p=12)
    plt.xlabel("Sample Index or (Cluster Size)")
    plt.ylabel("Distance")

plt.tight_layout()
plt.savefig('hierarchical_dendrograms.png')
plt.show()
```



In [25]:
```
plt.figure(figsize=(10, 5))

dendrogram(Z, p=15, truncate_mode='lastp', leaf_rotation=90.,color_threshold =
plt.show()
```



In [ ]: